验证方法：模拟无人机发送 JSON 数据，监控各节点日志。性能指标包括传输延迟（平均<100ms）、丢包率（<0.5%在 10km 范围内）、数据完整性（CRC 校验通过率>99%）。在实际部署中，该方案与 ATK-MO1218 定位模块协同，实现厘米级坐标的实时中转，支持非法无人机轨迹上报。

该方案的实现成本低、部署灵活，可进一步优化为无工具依赖的自定义程序（如 Python 脚本），增强与香橙派 AIpro 20T 的集成，实现全链条边缘计算。

3.3 智能监管平台架构与实现

智能监管平台是“空域智眸”系统的指挥控制与态势感知中枢。它不仅是前端多模态感知数据（身份广播、视觉识别）的汇聚点，更是执行数据融合、智能分析、实时告警与可视化决策的核心。为应对城市低空环境的复杂性与动态性，平台在设计上兼顾了高实时性、高并发性与高可扩展性，为整个监管闭环提供了坚实的技术支撑。

3.3.1 平台总体架构

平台采用面向服务的微服务架构思想进行设计，实现了前后端分离，确保各模块职责清晰、易于维护与独立扩展。整体上，平台遵循经典五层架构：

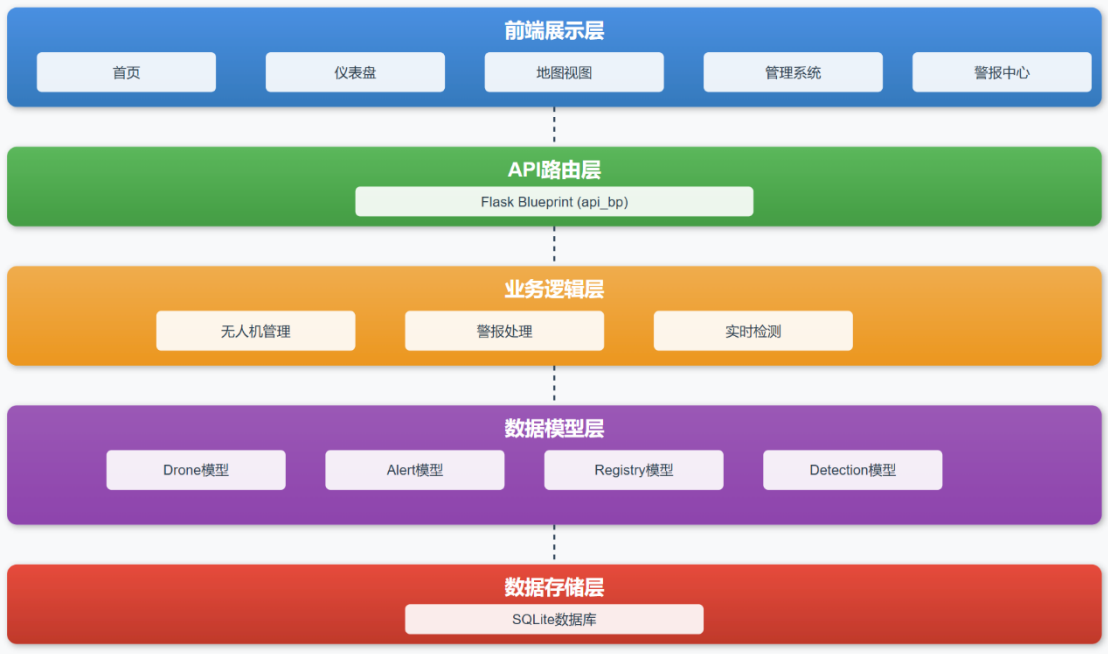


图 3.12 平台系统框架图

前端展示层 (Presentation Layer): 直接面向监管人员的用户界面，基于 HTML5、Web 技术构建 。该层集成了 Chart.js 图表库和自定义的地图引擎，负责将后端处理后的无人机位置、轨迹、状态和告警信息进行动态、直观的可视化展示。

API 路由层：以 api_bp 作为核心 Blueprint，按功能模块拆分路由子集，实现 “模块化路由管理”，便于后续扩展与维护。具体包含：

基础路由模块：负责系统健康检查、版本信息等通用接口；

实时数据路由模块：对接前端感知设备（视觉识别、LoRa 身份广播）的数据传输；

业务管理路由模块：涵盖无人机注册、检测记录、告警处理等核心业务；

实时通信路由模块：基于 SocketIO 实现服务器与前端的双向实时推送。

模块	路由端点	方法	功能描述	技术实现
基础路由	/api/health	GET	系统健康状态检查（返回 CPU、内存使用率等）	Flask-RESTful Resource
实时数据	/api/realtime/lora	POST	接收 LoRa 模块上传的无人机身份与 GNSS 坐标数据	Flask-RESTful + 数据校验
实时数据	/api/realtime/vision	POST	接收 YOLOv5 识别的无人机视觉坐标与特征信息	Flask-RESTful + 二进制数据解析
无	/api/dro	GET	查询所有注册无	Flask-SQLAlchemy

模块	路由端点	方法	功能描述	技术实现
人机管理	nes		人机信息（支持分页、筛选）	查询
无人机管理	/api/drones/<id>	PUT	更新指定无人机的注册信息（如归属、用途）	Flask-RESTful + 权限校验
检测记录	/api/detections	POST	存储视觉识别的无人机检测记录（关联注册信息匹配结果）	事务性数据库操作
告警管理	/api/alerts	GET	查询告警列表（支持按级别、时间筛选）	联合查询 Alert 与 Drone 表
告警管理	/api/alerts/<id>/solve	PUT	标记告警为“已处理”并记录处理日志	状态更新 + 日志记录
实时	/socket.io	-	建立 WebSocket 连接，推送无人	Flask-SocketIO 事件（如

模块	路由端点	方法	功能描述	技术实现
通信			机位置更新、新告警等实时消息	location_update）

表 3.1 API 设计

其中通过 Blueprint 将不同业务路由分离，避免单一文件冗余，符合微服务架构设计；基于 Flask-RESTful 定义请求参数、响应格式（如统一返回{"code": 200, "data": {...}}），便于前后端协作；SocketIO 路由支持事件驱动的实时推送，延迟控制在毫秒级，满足动态监管需求；预留 /api/v2 版本路由前缀，便于后续功能迭代时兼容旧接口。

业务逻辑层 (Business Logic Layer): 平台的核心，基于 Flask 框架实现 。此层不仅处理常规的 Web 请求和业务规则，更重要的是内嵌了智能分析引擎。该引擎实时执行“地理信息核验-身份特征绑定” 的核心匹配算法，持续比对来自视觉识别模块与身份广播模块的数据流，并根据预设规则（如禁飞区闯入、身份不匹配）驱动告警处理工作流 。

数据模型层 (Data Access Layer): 采用 Flask-SQLAlchemy 作为 ORM（对象关系映射）工具，为上层业务逻辑提供统一、标准化的数据访问接口 。该层有效屏蔽了底层数据库的差异，并通过内置的数据缓存与查询优化机制提升数据交互性能 。

数据存储层 (Data Storage Layer): 采用 SQLite 关系型数据库存储核心业务数据，包括无人机注册信息、实时状态、历史检测记录和告警日志等 。文件系统则用于存储配置文件与系统日志 。

3.3.2 核心技术栈选型

为了实现上述架构并满足性能要求，我们精心选择了一套成熟、高效且与项目技术生态高度契合的技术栈。



图 3.13 技术栈

后端技术栈:

Web 框架 (Flask): 选择 Flask 2.3.3，因其轻量级、高扩展性的特点，便于与本项目的 Python AI 算法模块（YOLOv5, JanusPro）进行无缝集成 。

实时通信 (Flask-SocketIO): 这是保障平台实时性的关键技术。通过 WebSocket 协议，服务器可主动向客户端推送毫秒级的状态更新与告警通知，满足动态监管的即时性需求 。

API 构建 (Flask-RESTful): 用于快速构建标准化、无状态的 RESTful API，实现了前后端的彻底解耦，便于团队并行开发与未来系统对接 。

数据库迁移 (Flask-Migrate): 用于管理数据库的版本演进，确保数据模型的变更在不同环境中（开发、测试、生产）的一致性 。

前端技术栈:

数据可视化 (Chart.js): 用于生成电池电量、信号强度等关键指标的动态图表，增强了数据的可读性 。

地图引擎 (SimpleMap): 自主封装的轻量级地图类，用于在页面上高效渲染无人机实时位置、历史轨迹及电子围栏等地理空间信息 。

开发与部署工具:

Web 服务器 (Nginx & Gunicorn): 在生产环境中，使用 Nginx 作为反向代理和负载均衡器，Gunicorn 作为 WSGI 服务器部署 Flask 应用，构成了高性能、高并发的处理架构 。

容器化 (Docker): 整个应用及依赖环境被封装在 Docker 容器中，实现了“一次构建，处处运行”，极大简化了部署流程并保障了环境一致性 。

3.3.3 核心功能模块设计

平台的功能设计紧密围绕无人机监管的实际业务流程，从数据流入到最终决策形成闭环。

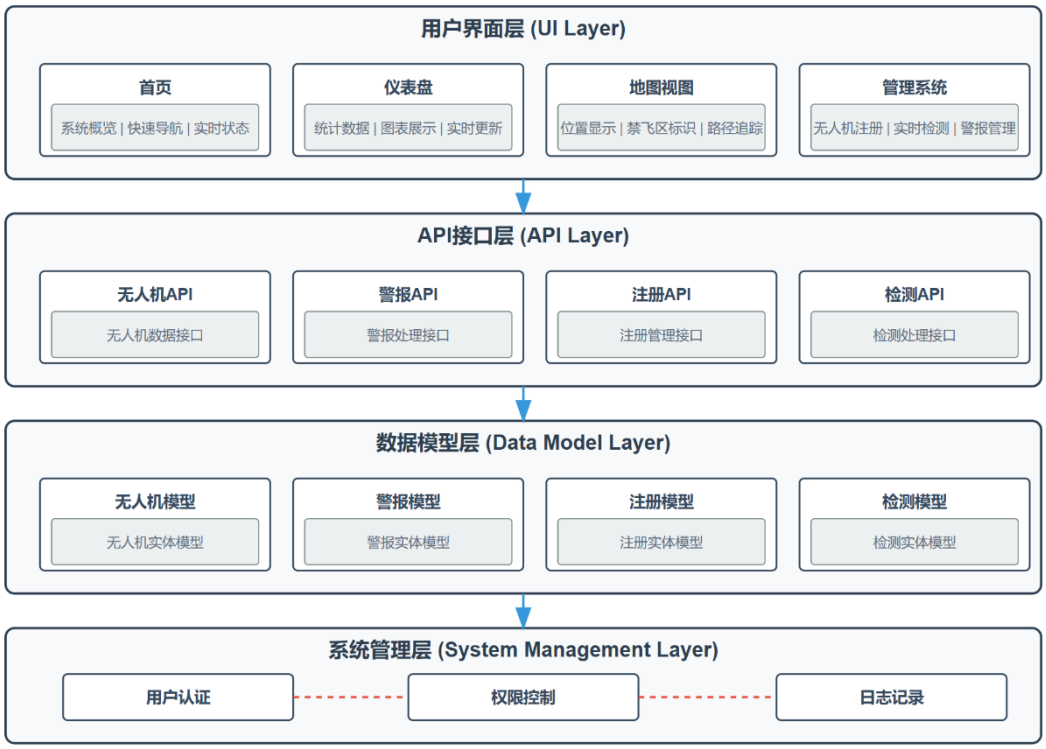


图 3.14 功能模块设计图

1. 用户界面与态势感知模块: 这是系统的“眼睛”。主仪表盘以卡片式布局实时展示活跃无人机数量、系统健康度、最新告警等核心指标 。地图视图是核心，它不仅能实时追踪所有无人机的位置，还能通过不同颜色或图标

明确区分“白名单”（身份已核验）与“不明目标”（仅视觉发现），并支持历史轨迹回放与禁飞区叠加显示 。

2. 实时监控与数据融合模块: 作为平台的“大脑”，该模块通过 API 接口和

SocketIO 通道，持续接收前端模块的数据：

LoRa 身份数据流：包含无人机唯一识别码、GNSS 厘米级坐标等。

视觉识别数据流：包含 YOLOv5 检测出的无人机二维坐标，以及经过 JanusPro 二次比对后的特征信息。 该模块实时对这两路数据进行时空对齐与关联匹配，并将融合结果（如“匹配成功”、“仅发现视觉目标”等）更新至数据库，并推送给前端。

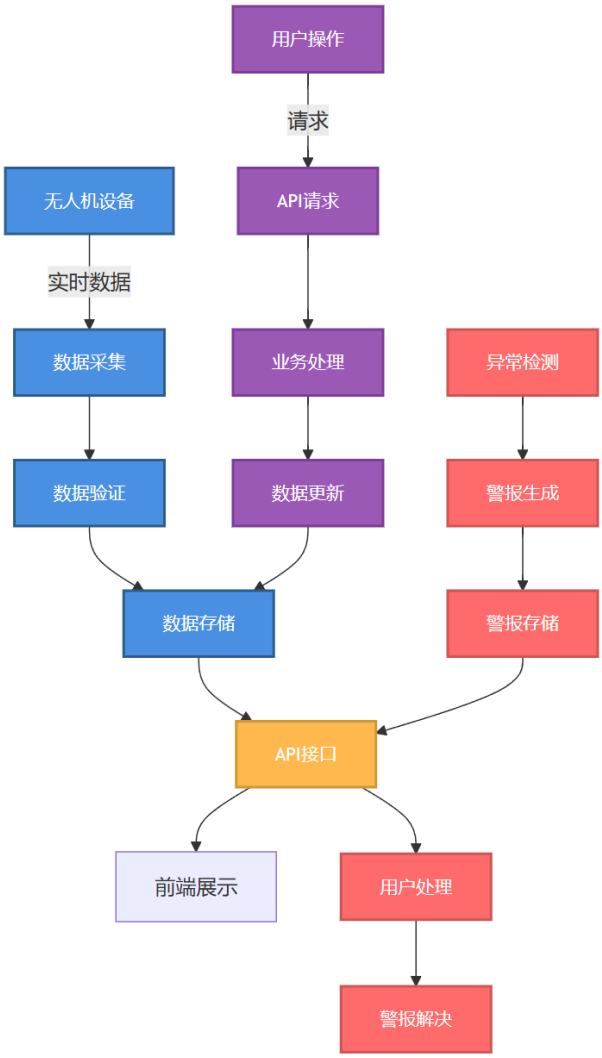


图 3.15 数据流程图

3. 智能警报管理模块：平台的“警报器”。当数据融合模块判定出现异常（如身份不匹配的“黑飞”无人机）或无人机行为触发预设规则（如低于安全高度、闯入电子围栏）时，此模块被激活 。它能自动生成包含时间、位置、快照等信息的警报，按“紧急”、“重要”、“一般”进行分级 ，并通过 UI 弹窗、声音、邮件等方式通知监

管员，形成从生成到闭环的完整处理流程 。

4. 注册与数据管理模块: 系统的“档案库”。提供对“白名单”无人机的注册、信息变更、注销等全生命周期管理功能 。同时，所有历史检测记录、飞行轨迹、告警日志均被持久化存储，支持按时间、区域、无人机 ID 等多维度进行查询、统计与导出，为事故追溯和优化运营提供数据支撑 。

3.3.4 数据模型设计

为了有效支撑上述功能，我们设计了一套规范化的关系型数据模型。模型的核心在于通过表间关联，建立起“静态注册信息”与“动态感知数据”的连接。

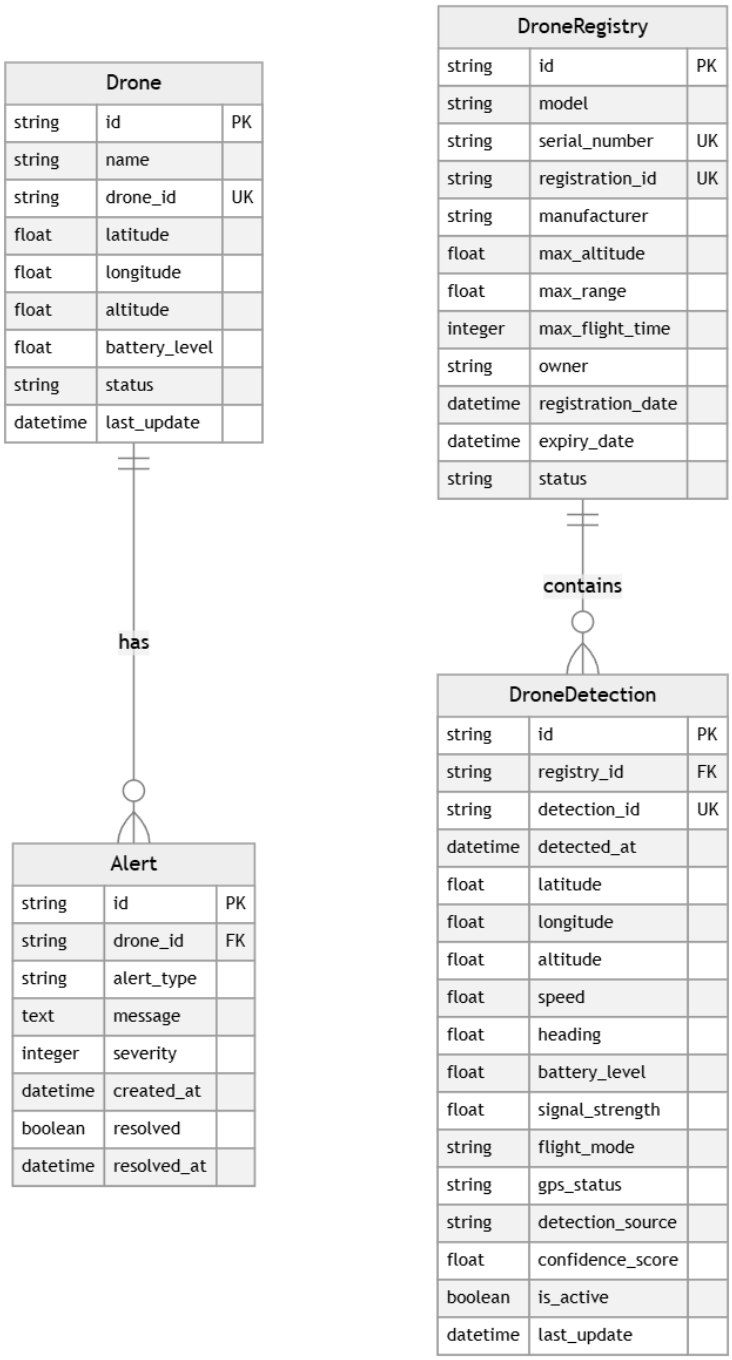


图 3.16 数据库关系图

无人机注册信息表 (DroneRegistry): 存储合法无人机的权威档案，serial_number（序列号）字段为唯一约束，是身份核验的基准 。

无人机检测记录表 (DroneDetection): 记录每一次由视觉系统捕获到的无人机事

件，包含时间、坐标、置信度等信息 。该表通过外键

registry_id 与注册表关联，若关联成功，则表明该无人机为“白名单”内设备 。

无人机实时状态表 (Drone): 主要用于存储通过 LoRa 广播上报的合法无人机实时状态，与注册表为一对一关联 。

警报管理表 (Alert): 当 DroneDetection 记录无法在 DroneRegistry 中找到匹配，或发生其他违规事件时，系统会在此表创建一条警报，并关联对应的无人机 ID 或检测 ID，以便追溯 。

该数据模型的设计，将“身份-位置”的绑定逻辑固化在数据库层面，为上层业务的快速、准确判断提供了结构化保障。

3.3.5 系统部署与运维

一个稳健的监管系统离不开科学的部署与运维策略。我们设计了一套支持高可用的分布式部署架构。



图 3.17 系统部署框架图

部署架构: 采用分层部署模式。前端由 Nginx 集群提供静态资源服务和反向代理 。应用层由多个运行在

Gunicorn 上的 Flask 应用实例组成服务集群，实现负载均衡 。数据层则采用主从复制的数据库集群和 Redis 缓存服务，确保数据的高可用和快速访问 。

3.3.6 系统展示

该图显示了监管平台的整体仪表盘，包括左侧的无人机列表（展示活跃无人机状态）和中央地图视图（实时标记无人机位置）。此界面用于快速监控空域态势，支持点击查看详情。

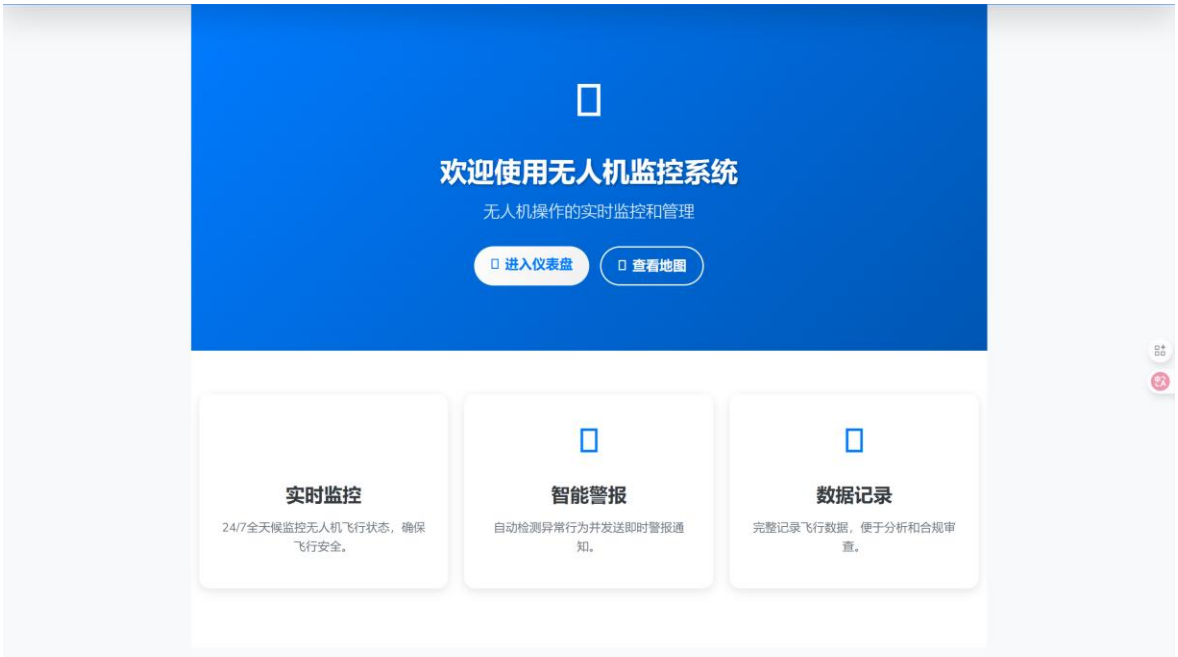


图 3.18 系统演示图

该图展示了无人机数据库的详细信息，包括 ID、型号、电量、位置坐标等信息。面板支持实时刷新，用于深度分析单个目标的合规性和行为。

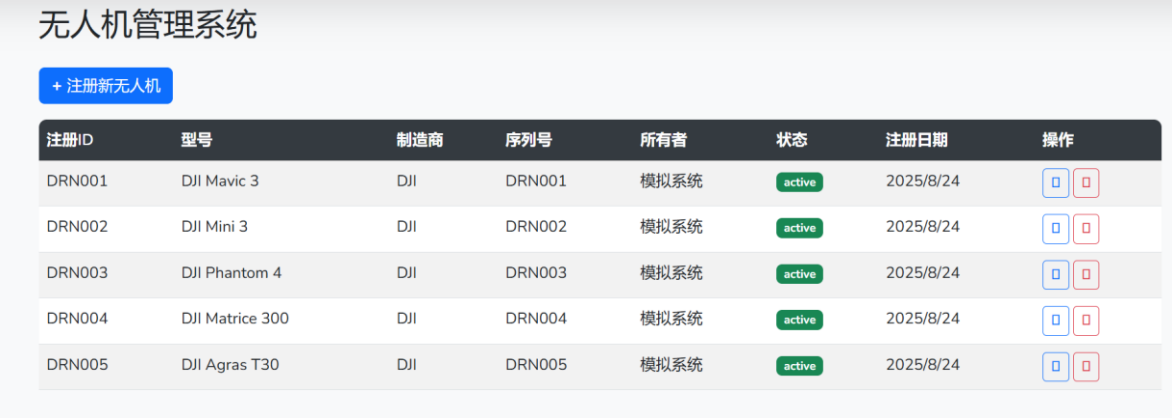


图 3.19 数据库管理图

该图为地图视图。它将前端采集到的海量、多模态数据，转化为了让监管人员一目了然的、可实时决策的空域态势情报。整个界面主要由三部分构成：左侧是实时无人机列表和详细信息面板，中间是多维空域态势图。

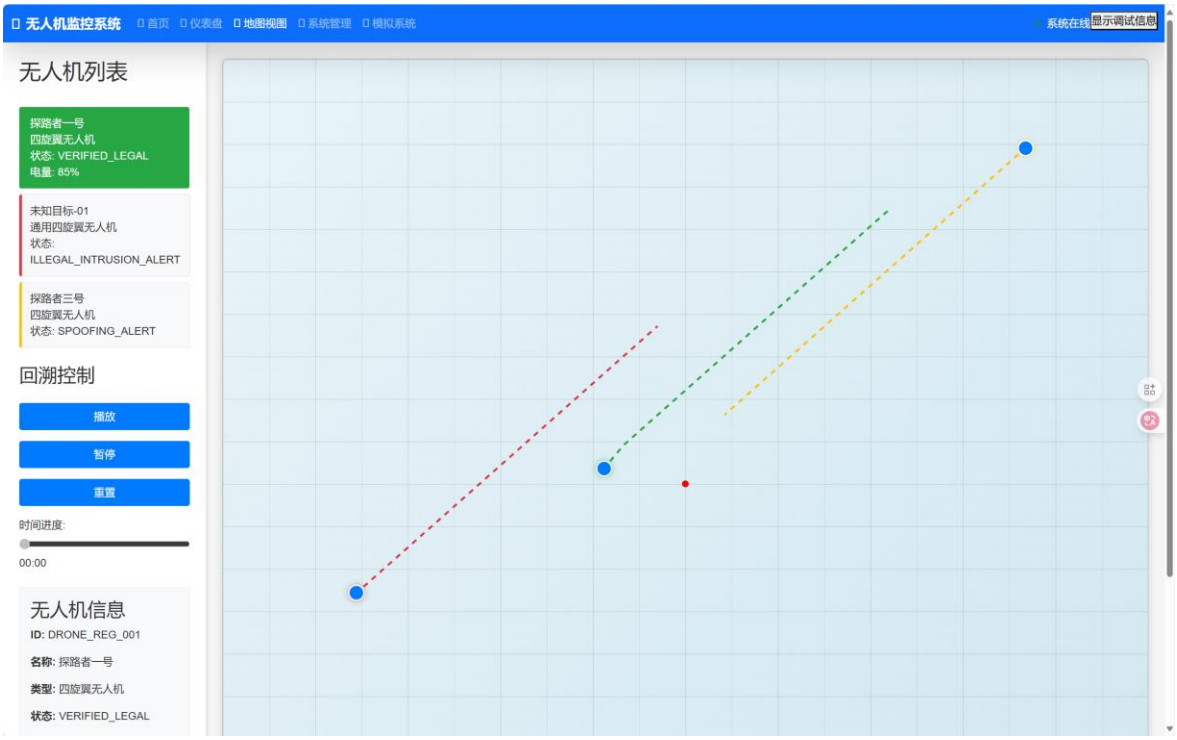


图 3.20 地图视角演示图